Pertemuan 6

Login & Logout · SharedPreferences · Splash Screen · Fragment

YANG AKAN KITA PELAJARI

Di pertemuan ini kita akan belajar untuk:

1. Membuat alur login dan logout, lengkap dengan dialog konfirmasi.
2. Menyimpan data kecil di HP dengan `SharedPreferences`, supaya pengguna tidak perlu login terus.
3. Membuat Splash Screen yang sekaligus mengecek apakah pengguna sudah login.
4. Memahami bedanya Activity dan Fragment.
5. Menampilkan beberapa Fragment dalam satu Activity.

YANG PERLU DISIAPKAN

- Project Pertemuan 5 (`com.example.irgi_tic`) masih bisa dijalankan tanpa error.
- *View Binding* sudah aktif, seperti di pertemuan-pertemuan sebelumnya.
- Emulator atau HP Android sudah siap dipakai.

ALUR MODUL

Tahap	Topik	Teori	Praktikum	Hasil akhir
0	Persiapan project	–	Struktur folder, tombol di `MainActivity`	Project siap
1	Login & Logout	`AlertDialog`, `Intent`, `finish()`	Layout login, logika login, logout	Login dan logout berjalan
2	SharedPreferences	Penyimpanan key-value	Simpan, baca, hapus `isLogin`	Login tersimpan setelah aplikasi ditutup
3	Splash Screen	Fungsi splash, coroutine `delay`	Splash dan alur Splash → Login/Main	Aplikasi punya layar pembuka
4	Fragment	Activity vs Fragment, `FragmentManager`	3 Fragment di `SixthActivity`	Pindah halaman tanpa Activity baru
5	Latihan mandiri	–	4 soal latihan	Tugas dikumpulkan

CARA MENGIKUTI MODUL

Penanda	Arti
TEORI	Penjelasan konsepnya. Baca dulu sebelum mulai praktik.
PRAKTIKUM	Langkah-langkah yang dikerjakan di Android Studio. Ikuti urutannya.
CEK HASIL	Daftar singkat di akhir praktik. Kalau semuanya sudah sesuai, baru lanjut ke tahap berikutnya.

---

0 Persiapan Project

PRAKTIKUM

Kita lanjutkan project dari Pertemuan 5, jadi **tidak perlu membuat project baru**. Buat package `pertemuan_6` yang sejajar dengan `pertemuan_5`, `LoginActivity`, dan `MainActivity`. Setelah itu buat satu activity baru, `SplashScreenActivity`, di package utama.

```
com.example.irgi_tic
  pertemuan_5
    LimaActivity
    WebViewActivity
  pertemuan_6
    SixthActivity
    SatuFragment
    DuaFragment
    TigaFragment
  LoginActivity
  MainActivity
  SplashScreenActivity
res
  layout
    activity_splash_screen.xml
    activity_login.xml
    activity_sixth.xml
    fragment_satu.xml
    fragment_dua.xml
    fragment_tiga.xml
  menu
    main_menu.xml
  drawable
    ic_arrow_back.xml
```

CATATAN File yang berlatar warna adalah file baru di pertemuan ini. Kalau membuat Activity lewat New → Activity → Empty Views Activity, otomatis ia terdaftar di AndroidManifest. Ketiga Fragment baru kita buat nanti di Tahap 4.

0.1 Tambahkan tombol di activity_main.xml

Taruh dua tombol ini di bawah tombol `btnToLima` dari Pertemuan 5. Tombol Logout belum berfungsi dulu, kita isi di Tahap 1.

XML

```xml
<Button
    android:id="@+id/btnToSixth"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:backgroundTint="#00796B"
    android:text="Pertemuan 6"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/btnToLima" />

<Button
    android:id="@+id/btnLogout"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:backgroundTint="#D32F2F"
    android:text="Logout"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/btnToSixth" />
```

0.2 Hubungkan tombol ke SixthActivity

Di `MainActivity`, tulis tepat di bawah listener `btnToLima`:

KOTLIN

```kotlin
binding.btnToSixth.setOnClickListener {
    startActivity(Intent(this@MainActivity, SixthActivity::class.java))
}
```

PERHATIAN Kalau `SixthActivity` masih merah, tekan Alt + Enter untuk menambahkan import-nya.

CEK HASIL **Sebelum lanjut, cek dulu:**

- Struktur foldernya sama seperti gambar di atas.
- Aplikasi bisa dijalankan tanpa error, dan tombol “Pertemuan 6” serta “Logout” muncul di `MainActivity`.
- Kalau “Pertemuan 6” ditekan, terbuka halaman kosong (`SixthActivity`).

1 Login & Logout

TEORI

Alur login yang kita buat

Halaman login menerima username dan password, lalu memeriksanya. Kalau cocok, pengguna masuk ke halaman utama. Kalau tidak, muncul pesan bahwa login gagal. Supaya mudah dicoba, aturannya kita buat sederhana: **login berhasil kalau username sama persis dengan password**.

AlertDialog

`AlertDialog` adalah kotak kecil yang muncul di tengah layar untuk memberi pesan atau meminta konfirmasi dari pengguna. Cara membuatnya lewat `AlertDialog.Builder`, dengan bagian-bagian berikut.

Fungsi	Keterangan
`setTitle("...")`	Judul dialog
`setMessage("...")`	Isi pesan
`setPositiveButton("Ya") { dialog, _ → ... }`	Tombol aksi utama (misalnya Ya / OK)
`setNegativeButton("Batal") { dialog, _ → ... }`	Tombol pembatalan
`show()`	Menampilkan dialog. Tanpa baris ini dialog tidak akan muncul

Pindah halaman dan finish()

Untuk pindah halaman kita pakai `Intent`, yaitu `startActivity(Intent(this, Tujuan::class.java))`. Sedangkan `finish()` menutup Activity yang sedang tampil, sehingga halaman itu **tidak bisa dibuka lagi lewat tombol back**. Itu sebabnya setelah login berhasil kita panggil `finish()`: kita tidak ingin pengguna menekan back lalu kembali ke halaman login.

PRAKTIKUM

1.1 Siapkan layout activity_login.xml

Komponen	ID	Keterangan
ImageView	`ivLogo`	Logo aplikasi di bagian atas
EditText	`etUsername`	Input username
EditText	`etPassword`	Input password, `inputType="textPassword"`
Button	`btnLogin`	Tombol Login

```xml
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/main"
```

```
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    android:padding="32dp">

    <ImageView
        android:id="@+id/ivLogo"
        android:layout_width="120dp"
        android:layout_height="120dp"
        android:contentDescription="Logo aplikasi"
        android:src="@mipmap/ic_launcher" />

    <EditText
        android:id="@+id/etUsername"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="32dp"
        android:hint="Username"
        android:inputType="text" />

    <EditText
        android:id="@+id/etPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Password"
        android:inputType="textPassword" />

    <Button
        android:id="@+id/btnLogin"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
        android:text="Login" />
</LinearLayout>
```

CATATAN Kalau `activity_login.xml` sudah ada dari pertemuan sebelumnya, cukup pastikan id-nya sama dengan tabel di atas. Logo `@mipmap/ic_launcher` boleh diganti dengan logo kalian sendiri.

1.2 Tulis logika login di LoginActivity

KOTLIN

```kotlin
class LoginActivity : AppCompatActivity() {
    private lateinit var binding: ActivityLoginBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityLoginBinding.inflate(layoutInflater)
        setContentView(binding.root)

        binding.btnLogin.setOnClickListener {
            val username = binding.etUsername.text.toString().trim()
            val password = binding.etPassword.text.toString()

            if (username.isNotEmpty() && username == password) {
                startActivity(Intent(this, MainActivity::class.java))
                finish()
            } else {
```

```
                AlertDialog.Builder(this)
                    .setTitle("Login Gagal")
                    .setMessage("Silahkan coba lagi")
                    .setPositiveButton("OK") { dialog, _ -> dialog.dismiss() }
                    .show()
            }
        }
    }
}
```

PERHATIAN Saat memilih import untuk `AlertDialog`, pilih yang `androidx.appcompat.app.AlertDialog`.

1.3 Tambahkan fitur Logout di MainActivity

Sekarang bagian logout. Saat `btnLogout` ditekan, munculkan dialog konfirmasi. Kalau pengguna memilih "Ya", halaman ditutup dengan `finish()`.

KOTLIN

```kotlin
binding.btnLogout.setOnClickListener {
    AlertDialog.Builder(this)
        .setTitle("Logout")
        .setMessage("Apakah Anda yakin ingin keluar?")
        .setPositiveButton("Ya") { dialog, _ ->
            dialog.dismiss()
            finish()
        }
        .setNegativeButton("Batal") { dialog, _ -> dialog.dismiss() }
        .show()
}
```

CEK HASIL **Sebelum lanjut, cek dulu:**

- Kalau username dan password berbeda, muncul dialog "Silahkan coba lagi".
- Kalau sama, kamu pindah ke `MainActivity`, dan tombol back tidak membawamu kembali ke login.
- Tombol Logout memunculkan dialog konfirmasi. "Batal" menutup dialog, "Ya" menutup halaman.

2 SharedPreferences

TEORI

Kenapa perlu menyimpan data?

Dengan aplikasi yang sekarang, setiap kali aplikasi ditutup lalu dibuka lagi, pengguna harus login ulang. Ini terjadi karena data di memori ikut hilang saat aplikasi ditutup. Kita perlu tempat untuk menyimpan data kecil yang **tetap ada** walaupun aplikasinya ditutup.

Apa itu SharedPreferences?

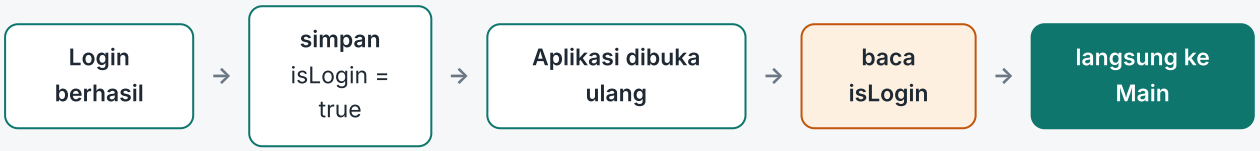
`SharedPreferences` adalah penyimpanan sederhana di dalam aplikasi yang bentuknya pasangan *key-value* (kunci dan nilainya). Cocok untuk data kecil, misalnya:

- Status login (`isLogin` bernilai `true` atau `false`)
- Username pengguna
- Pilihan tema (gelap atau terang)

PERHATIAN Jangan simpan password asli di SharedPreferences. Di praktikum ini kita hanya menyimpan status login dan username.

Fungsi	Keterangan
`getSharedPreferences("user_pref", MODE_PRIVATE)`	Membuka file penyimpanan bernama `user_pref`, yang hanya bisa diakses oleh aplikasi ini
`edit()`	Menyiapkan editor untuk mengubah data
`putString("key", "value")` / `putBoolean("key", true)`	Menyimpan data sesuai tipenya
`apply()`	Menyimpan perubahan di latar belakang (ini yang kita pakai)
`getString("key", "")` / `getBoolean("key", false)`	Membaca data. Nilai kedua adalah nilai cadangan kalau key-nya belum ada
`remove("key")` / `clear()`	Menghapus satu key / seluruh data

Alurnya untuk status login



Saat logout, datanya kita hapus dengan `clear()`, jadi ketika aplikasi dibuka lagi pengguna kembali melihat halaman login.

PRAKTIKUM

2.1 Cek isLogin di LoginActivity

Kalau `isLogin` bernilai `true`, pengguna langsung kita arahkan ke `MainActivity`. Ubah bagian awal `onCreate()` di `LoginActivity` menjadi seperti ini:

KOTLIN

```kotlin
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    // Kode ini harus selalu dipanggil saat butuh akses "user_pref"
    val sharedPref = getSharedPreferences("user_pref", MODE_PRIVATE)

    // Kondisi jika isLogin bernilai true
    val isLogin = sharedPref.getBoolean("isLogin", false)
    if (isLogin) {
        startActivity(Intent(this, MainActivity::class.java))
        finish()
        return
    }

    binding = ActivityLoginBinding.inflate(layoutInflater)
    setContentView(binding.root)
    binding.btnLogin.setOnClickListener { ... }
}
```

PERTANYAAN Coba pikirkan: kenapa pengecekan `isLogin` ditaruh **sebelum** `setContentView()`, dan kenapa ada `return` setelah `finish()`? Tulis jawabanmu dengan kata-katamu sendiri.

2.2 Simpan isLogin dan username saat login berhasil

KOTLIN

```kotlin
binding.btnLogin.setOnClickListener {
    val username = binding.etUsername.text.toString().trim()
    val password = binding.etPassword.text.toString()

    if (username.isNotEmpty() && username == password) {
        val editor = sharedPref.edit()
        editor.putBoolean("isLogin", true)
        editor.putString("username", username)
        editor.apply()

        startActivity(Intent(this, MainActivity::class.java))
        finish()
    } else {
        ... // AlertDialog “Silahkan coba lagi”
    }
}
```

2.3 Hapus data saat Logout

Di `MainActivity`, buka lagi `user_pref`, lalu hapus datanya di dalam `setPositiveButton`.

```kotlin
binding.btnLogout.setOnClickListener {
    val sharedPref = getSharedPreferences("user_pref", MODE_PRIVATE)

    AlertDialog.Builder(this)
        .setTitle("Logout")
        .setMessage("Apakah Anda yakin ingin keluar?")
        .setPositiveButton("Ya") { dialog, _ ->
            val editor = sharedPref.edit()
            editor.clear()
            editor.apply()
            dialog.dismiss()
            finish()
        }
        .setNegativeButton("Batal") { dialog, _ -> dialog.dismiss() }
        .show()
}
```

Kalau hanya ingin menghapus satu key saja, pakai `remove()`:

```kotlin
editor.remove("isLogin")
```

PERHATIAN Selain lewat `remove()` atau `clear()`, data SharedPreferences hanya bisa hilang kalau kita memilih “Clear Data” untuk aplikasi ini di pengaturan HP.

2.4 Test Drive

No	Langkah	Hasil yang diharapkan
1	Login dengan username dan password yang sama	Masuk ke `MainActivity`
2	Tutup (destroy) aplikasi, lalu buka kembali	Langsung ke `MainActivity` karena `isLogin` masih `true`
3	Klik `btnLogout`, pilih “Ya”	Data SharedPreferences terhapus
4	Buka kembali aplikasi	Tampil halaman login

CEK HASIL **Sebelum lanjut, cek dulu:**

- Keempat langkah test drive berjalan seperti di tabel.
- Kamu sudah bisa menjawab pertanyaan di langkah 2.1.

3 Splash Screen

TEORI

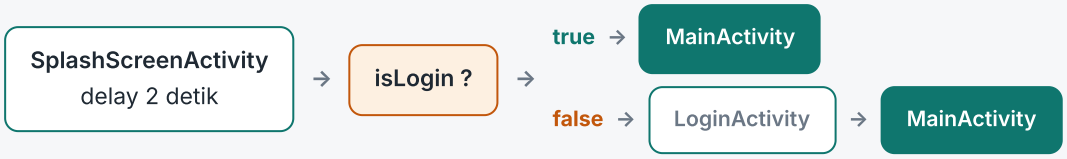
Splash Screen adalah layar pembuka yang muncul sebentar ketika aplikasi baru dijalankan, sebelum halaman utama tampil. Biasanya dipakai untuk:

- Menampilkan logo atau branding aplikasi
- Menjalankan proses awal, misalnya mengambil data atau **mengecek apakah pengguna sudah login**

Menahan layar dengan coroutine

Supaya splash tampil beberapa detik, kita pakai `lifecycleScope.launch { delay(2000) }`. `delay()` menunda tanpa membuat layar macet (berbeda dengan `Thread.sleep()`), dan `lifecycleScope` otomatis berhenti kalau Activity-nya ditutup. Angka 2000 adalah milidetik, jadi 2 detik. Anggap saja itu waktu untuk mengambil data.

Alur aplikasi setelah ada Splash



PRAKTIKUM

3.1 Jadikan SplashScreenActivity halaman pertama

Buka `AndroidManifest.xml`, lalu pindahkan `intent-filter` MAIN/LAUNCHER ke `SplashScreenActivity` supaya activity ini yang pertama dibuka.

XML

```xml
<activity
    android:name=".SplashScreenActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>

<activity
    android:name=".LoginActivity"
    android:exported="false" />
```

PERHATIAN Jangan lupa hapus `intent-filter` LAUNCHER dari activity yang lama (misalnya `LoginActivity`). Kalau tidak, aplikasimu akan punya dua ikon.

3.2 Desain activity_splash_screen.xml

- Atur warna background layar
- Pasang satu `ImageView` sebagai logo di tengah layar
- Tambahkan nama, slogan, atau versi aplikasi

```xml
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#0097A7">

    <ImageView
        android:id="@+id/ivLogo"
        android:layout_width="140dp"
        android:layout_height="140dp"
        android:contentDescription="Logo aplikasi"
        android:src="@mipmap/ic_launcher"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:id="@+id/tvAppName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Nama Aplikasi"
        android:textColor="@android:color/white"
        android:textSize="22sp"
        android:textStyle="bold"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/ivLogo" />

    <TextView
        android:id="@+id/tvVersion"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="32dp"
        android:text="Versi 1.0"
        android:textColor="@android:color/white"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

CATATAN Nama aplikasi, slogan, dan versi silakan diganti sesuai aplikasi kalian. Warna dan logonya juga bebas dikreasikan.

3.3 Tulis kode SplashScreenActivity

Kita mulai dari versi sederhana dulu: tunggu 2 detik, lalu langsung ke `MainActivity`. Tambahkan di `onCreate()`:

```kotlin
lifecycleScope.launch {
    delay(2000) // simulasi pengambilan data selama 2 detik
    val intent = Intent(this@SplashScreenActivity, MainActivity::class.java)
    startActivity(intent)
    finish()
}
```

CATATAN Import yang dibutuhkan: `androidx.lifecycle.lifecycleScope`, `kotlinx.coroutines.launch`, dan `kotlinx.coroutines.delay`. Kalau `lifecycleScope` masih merah, cek apakah dependency `lifecycle-runtime-ktx` sudah ada di `build.gradle`.

Jalankan aplikasinya. Splash akan tampil 2 detik, lalu masuk ke `MainActivity`. Di Android 12 ke atas, sistem menampilkan splash bawaan (ikon aplikasi) sebentar sebelum `SplashScreenActivity` muncul. Itu normal, tidak perlu dikhawatirkan.

3.4 Pindahkan pengecekan isLogin ke Splash

Sekarang ganti isi `lifecycleScope.launch` dengan kode ini:

```kotlin
lifecycleScope.launch {
    delay(2000)

    val sharedPref = getSharedPreferences("user_pref", MODE_PRIVATE)
    val isLogin = sharedPref.getBoolean("isLogin", false)

    val target = if (isLogin) MainActivity::class.java else LoginActivity::class.java
    startActivity(Intent(this@SplashScreenActivity, target))
    finish()
}
```

Karena pengecekan sudah dilakukan di Splash, **hapus blok pengecekan `isLogin` dari `onCreate()` di `LoginActivity`** (yang kita tulis di langkah 2.1). Bagian yang menyimpan `isLogin` dan `username` saat login berhasil tetap dipertahankan.

PERHATIAN `MainActivity` sekarang menjadi satu-satunya halaman yang tersisa, karena Splash dan Login sudah ditutup dengan `finish()`. Akibatnya, Logout (yang memakai `finish()`) akan menutup aplikasi, dan saat dibuka lagi alurnya Splash lalu Login. Di Latihan Mandiri nomor 3 kamu akan mengubahnya supaya Logout langsung membawa ke halaman login.

No	Skenario	Hasil
1	Pertama kali membuka aplikasi (belum login)	Splash → Login
2	Login berhasil, tutup lalu buka aplikasi	Splash → Main
3	Logout, lalu buka aplikasi	Splash → Login

CEK HASIL **Sebelum lanjut, cek dulu:**

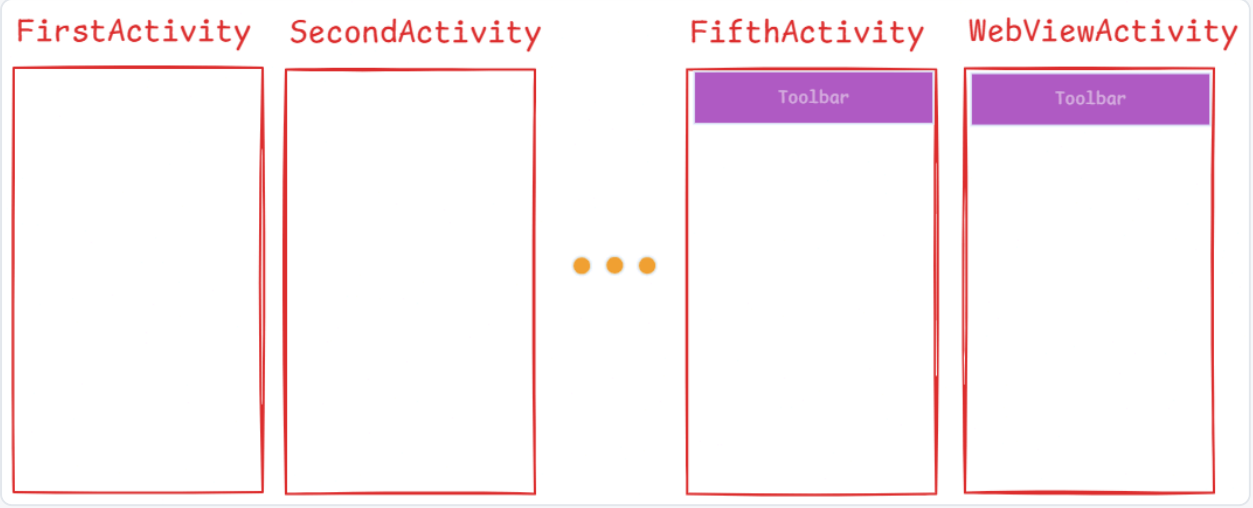
- Aplikasi hanya punya satu ikon di HP.
- Ketiga skenario di tabel menghasilkan hasil yang benar.

4 Fragment

TEORI

Activity: cara yang kita pakai selama ini

Sejauh ini, setiap kali butuh halaman baru kita membuat Activity baru lengkap dengan tampilannya sendiri.



Setiap halaman memiliki Activity dan UI sendiri-sendiri

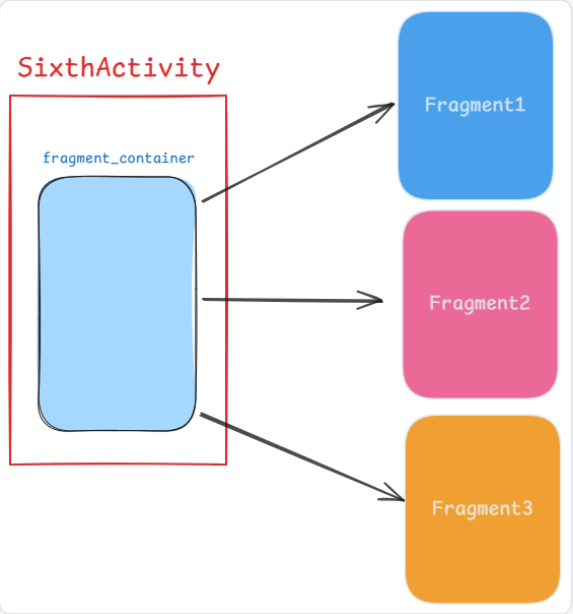
Cara ini punya beberapa kekurangan:

- Pindah antar Activity berarti Android harus memuat ulang seluruh tampilan. Kalau layarnya banyak (misalnya 5 Activity), perpindahan bisa terasa lambat dan memicu lag di HP berspesifikasi rendah.
- Kita tidak bisa menampilkan elemen yang tetap ada di semua layar, seperti sidebar atau bottom navigation. Aplikasi seperti YouTube atau Instagram memakai Fragment, makanya Bottom Navigation-nya tetap di tempat saat kita berpindah layar.

CATATAN Bukan berarti Activity jelek. Activity tetap cocok untuk aplikasi dengan 2–3 halaman, atau untuk halaman yang fungsinya benar-benar berbeda seperti `LoginActivity` dan `MainActivity`.

Fragment: bagian UI yang bisa dipakai ulang

Fragment adalah potongan tampilan yang hidup di dalam satu Activity. Activity cukup menyediakan sebuah *container*, lalu Fragment yang berbeda tinggal ditampilkan atau diganti di dalamnya.



SixthActivity dengan satu fragment_container yang diisi Fragment 1, 2, atau 3

Aspek	Activity	Fragment
Jumlah halaman	Satu halaman = satu Activity	Banyak halaman dalam satu Activity
Perpindahan	Activity dimuat ulang	Fragment dimuat atau dilepas sesuai kebutuhan
UI yang tetap (bottom navigation, sidebar)	Tidak bisa	Bisa, ada di Activity induk
Navigasi	`Intent`	Bottom Navigation, ViewPager, TabLayout, atau tombol

Komponen yang akan kita pakai

Komponen	Peran
`Fragment`	Kelas untuk satu "halaman kecil". Di `onCreateView()` layout-nya dipasang
`FrameLayout` (`fragment_container`)	Wadah di Activity tempat Fragment ditampilkan
`supportFragmentManager`	Pengelola Fragment di dalam Activity
`beginTransaction()` ... `commit()`	Satu paket perubahan Fragment (ganti, tambah, hapus)
`replace(containerId, fragment)`	Mengganti isi container dengan Fragment lain
`addToBackStack(null)`	Menyimpan transaksi supaya tombol back bisa kembali ke Fragment sebelumnya

PRAKTIKUM

4.1 Siapkan Toolbar dan tombol back

Toolbar dan tombol back dipasang dengan cara yang sama seperti di Pertemuan 5 (`ic_arrow_back` sudah ada di `res/drawable`). Kodenya nanti kita tulis lengkap di langkah 4.5.

4.2 Susun activity_sixth.xml

Isi layout dengan Toolbar, tiga Button (`btnFragment1`, `btnFragment2`, `btnFragment3`) tepat di bawah Toolbar, dan sebuah `FrameLayout` sebagai container. Pastikan constraint-nya terpasang dan `layout_height` container diatur `0dp`.

XML

```
<androidx.constraintlayout.widget.ConstraintLayout ...
    android:id="@+id/main"
    tools:context=".pertemuan_6.SixthActivity">

    <androidx.appcompat.widget.Toolbar
        android:id="@+id/toolbar"
        android:layout_width="0dp"
        android:layout_height="?attr/actionBarSize"
        android:background="#0097A7"
        app:subtitleTextColor="@android:color/white"
        app:titleTextColor="@android:color/white"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

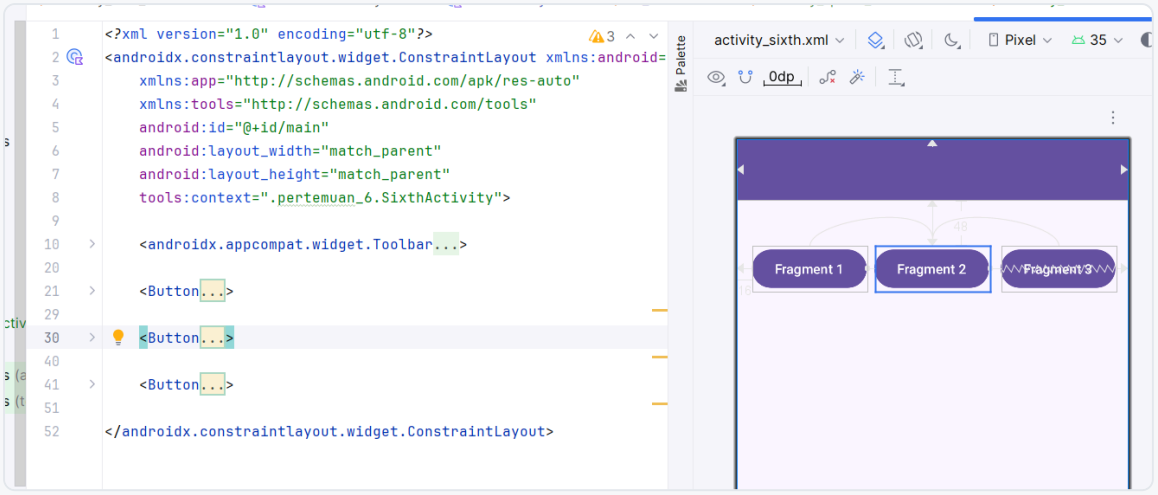
    <Button
        android:id="@+id/btnFragment1"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="16dp"
        android:layout_marginTop="16dp"
        android:layout_marginEnd="4dp"
        android:backgroundTint="#00796B"
        android:text="Fragment 1"
        app:layout_constraintEnd_toStartOf="@+id/btnFragment2"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/toolbar" />

    <Button
        android:id="@+id/btnFragment2"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="4dp"
        android:layout_marginEnd="4dp"
        android:backgroundTint="#00796B"
        android:text="Fragment 2"
        app:layout_constraintEnd_toStartOf="@+id/btnFragment3"
        app:layout_constraintStart_toEndOf="@+id/btnFragment1"
        app:layout_constraintTop_toTopOf="@+id/btnFragment1" />

    <Button
        android:id="@+id/btnFragment3"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="4dp"
        android:layout_marginEnd="16dp"
```

```
        android:backgroundTint="#00796B"
        android:text="Fragment 3"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@+id/btnFragment2"
        app:layout_constraintTop_toTopOf="@+id/btnFragment1" />

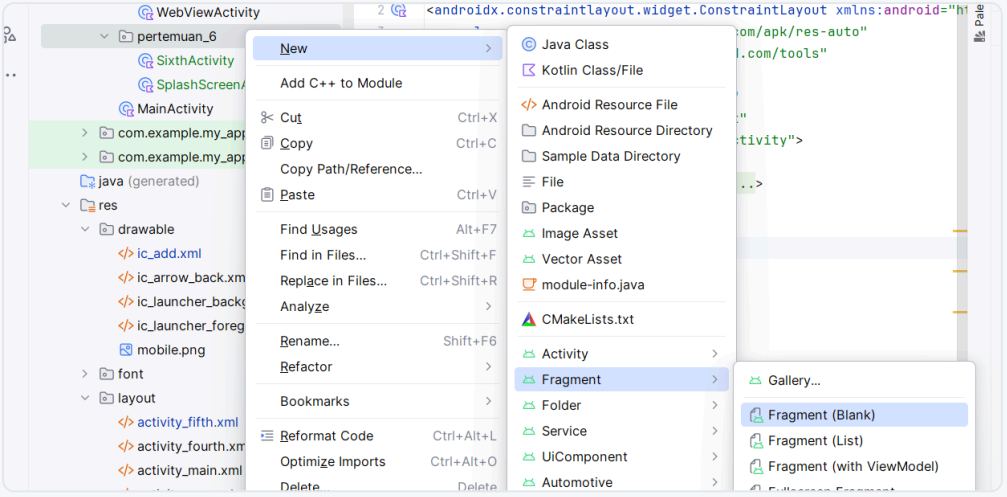
    <FrameLayout
        android:id="@+id/fragment_container"
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:layout_marginTop="20dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/btnFragment2" />
</androidx.constraintlayout.widget.ConstraintLayout>
```



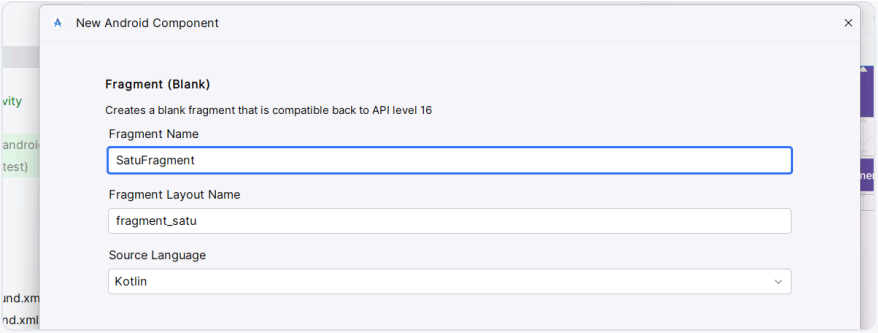
Tampilan activity_sixth.xml: Toolbar, tiga tombol, dan container di bawahnya

4.3 Buat tiga Fragment

Klik kanan package `pertemuan_6` → New → Fragment → Fragment (Blank).



Isi nama `SatuFragment` (layout `fragment_satu`), pilih Kotlin, lalu klik Finish.



Buka `SatuFragment`, lalu **hapus kode yang tidak diperlukan**. Yang disisakan cukup `onCreateView()` saja:

```kotlin
class SatuFragment : Fragment() {

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        return inflater.inflate(R.layout.fragment_satu, container, false)
    }
}
```

Lalu buka `fragment_satu.xml` dan ubah teks serta warna latarnya:

```xml
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#EFA2FF"
    tools:context=".pertemuan_6.SatuFragment">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:text="Ini Fragment 1"
        android:textSize="40sp"
        android:textStyle="bold" />
</FrameLayout>
```

Lakukan hal yang sama untuk `DuaFragment` dan `TigaFragment` (di `onCreateView()` pakai `R.layout.fragment_dua` dan `R.layout.fragment_tiga`):

Fragment	Layout	Teks	Warna latar (contoh)
`SatuFragment`	`fragment_satu.xml`	Ini Fragment 1	`#EFA2FF` (ungu muda)
`DuaFragment`	`fragment_dua.xml`	Ini Fragment 2	`#5B9CF5` (biru)
`TigaFragment`	`fragment_tiga.xml`	Ini Fragment 3	`#FFB255` (oranye)

4.4 Tulis fungsi replaceFragment()

Tambahkan fungsi ini di bawah `onCreate()` pada `SixthActivity`. Tugasnya mengganti isi container dengan Fragment yang kita kirim:

```kotlin
private fun replaceFragment(fragment: Fragment, addToBackStack: Boolean = true) {
    val transaction = supportFragmentManager.beginTransaction()
        .replace(binding.fragmentContainer.id, fragment)
    if (addToBackStack) transaction.addToBackStack(null)
    transaction.commit()
}
```

KOTLIN

PERHATIAN Fragment pertama sengaja dipasang dengan `addToBackStack = false`. Kalau fragment awal ikut masuk back stack, saat tombol back ditekan terakhir kali container akan tampil kosong dulu sebelum Activity tertutup.

4.5 Lengkapi SixthActivity

```kotlin
class SixthActivity : AppCompatActivity() {
    private lateinit var binding: ActivitySixthBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivitySixthBinding.inflate(layoutInflater)
        setContentView(binding.root)
        ...
        setSupportActionBar(binding.toolbar)
        supportActionBar?.apply {
            title = "Activity Sixth"
            subtitle = "Penerapan Fragment"
            setDisplayHomeAsUpEnabled(true)
            setDisplayShowHomeEnabled(true)
            setHomeAsUpIndicator(R.drawable.ic_arrow_back)
        }

        // Menampilkan fragment pertama secara default
        replaceFragment(SatuFragment(), addToBackStack = false)

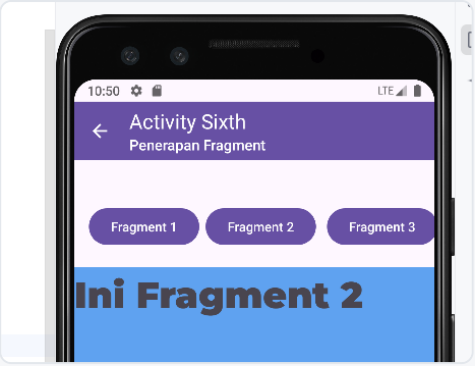
        // Setup event click untuk mengganti fragment
        binding.btnFragment1.setOnClickListener { replaceFragment(SatuFragment()) }
        binding.btnFragment2.setOnClickListener { replaceFragment(DuaFragment()) }
        binding.btnFragment3.setOnClickListener { replaceFragment(TigaFragment()) }
    }

    private fun replaceFragment(fragment: Fragment, addToBackStack: Boolean = true) { ... }

    override fun onOptionsItemSelected(item: MenuItem): Boolean {
        return when (item.itemId) {
            android.R.id.home -> {
                onBackPressedDispatcher.onBackPressed()
                true
            }
            else -> super.onOptionsItemSelected(item)
        }
    }
}
```

KOTLIN

4.6 Test Drive



Hasil akhir: SixthActivity dengan tombol pengganti fragment

CEK HASIL

Sebelum lanjut, cek dulu:

- `SatuFragment` langsung tampil begitu halaman dibuka.
- Tombol Fragment 1, 2, dan 3 mengganti isi container.
- Tombol back (di HP maupun di Toolbar) kembali ke fragment sebelumnya, lalu keluar dari halaman kalau sudah tidak ada lagi yang bisa dikembalikan.

5 Latihan Mandiri

Sekarang giliran kalian mengembangkan sendiri aplikasinya dengan menggabungkan materi Pertemuan 5 dan 6.

5.1 Sapaan pengguna

WAJIB

Tambahkan `TextView` di `MainActivity` yang menampilkan "Halo, (username)". Ambil key `username` dari SharedPreferences, dan beri nilai cadangan kalau datanya tidak ada.

```kotlin
val username = sharedPref.getString("username", "Pengguna")
```

5.2 Fragment menampilkan data tersimpan

WAJIB

Ubah `SatuFragment` supaya menampilkan username yang tersimpan. Di dalam Fragment, SharedPreferences diakses lewat `requireContext()`, lalu `TextView` diisi di `onViewCreated()`.

```kotlin
val sharedPref = requireContext().getSharedPreferences("user_pref", Context.MODE_PRIVATE)
val username = sharedPref.getString("username", "Pengguna")
```

CATATAN Beri id pada `TextView` di `fragment_satu.xml` (misalnya `tvNama`), lalu ambil dengan `view.findViewById<TextView>(R.id.tvNama)`.

5.3 Logout dari Option Menu

TANTANGAN

Tambahkan item "Logout" di `main_menu.xml` (dari Pertemuan 5) dengan `showAsAction="never"`, lalu pasang menunya di `SixthActivity` lewat `onCreateOptionsMenu()`. Saat dipilih, tampilkan dialog konfirmasi, hapus SharedPreferences, lalu kembali ke `LoginActivity` sambil membersihkan seluruh back stack:

```kotlin
val intent = Intent(this, LoginActivity::class.java)
intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
startActivity(intent)
```

5.4 Fragment keempat: WebView

TANTANGAN

Buat `WebFragment` yang menampilkan halaman web seperti `WebViewActivity` di Pertemuan 5, lalu tambahkan tombol `btnFragment4` di `activity_sixth.xml`. Cara memakai `WebViewClient`, `javaScriptEnabled`, dan `loadUrl()` sama seperti sebelumnya. Izin INTERNET sudah ada di manifest.

Checklist pengumpulan

- [ ] Alur Splash → Login → Main berjalan sesuai tabel skenario di langkah 3.4
- [ ] Status login tetap tersimpan setelah aplikasi ditutup, dan hilang setelah Logout

- [ ] `SixthActivity` menampilkan 3 fragment yang bisa diganti lewat tombol
- [ ] Latihan 5.1 dan 5.2 selesai

---

6 Pertanyaan Pemahaman

1. Untuk apa `finish()` dipanggil setelah `startActivity()` saat login berhasil?
2. Kenapa `isLogin` dibaca dengan nilai cadangan `false`?
3. Apa perbedaan `clear()` dan `remove("isLogin")`?
4. Kenapa Splash Screen cocok dipakai untuk mengecek status login?
5. Sebutkan dua keuntungan Fragment dibanding membuat Activity baru untuk setiap halaman.
6. Apa yang terjadi kalau `addToBackStack(null)` tidak dipanggil di `replaceFragment()`?

---

7 Kalau Ada yang Error

Gejala	Kemungkinan penyebab	Solusi
`ActivityLoginBinding` atau `ActivitySixthBinding` merah	View Binding belum aktif atau nama layout berbeda	Pastikan `viewBinding = true` di `build.gradle`, nama layout sesuai, lalu Sync dan Rebuild
Muncul dua ikon aplikasi	`intent-filter` LAUNCHER masih ada di activity lama	Hapus dari activity lama, sisakan di `SplashScreenActivity`
Aplikasi langsung berhenti saat pindah halaman	Activity belum terdaftar di manifest	Tambahkan `<activity>` di `AndroidManifest.xml`
`lifecycleScope` / `delay` tidak dikenali	Import atau dependency belum ada	Tambahkan import, dan dependency `lifecycle-runtime-ktx`
Dialog tidak muncul	Lupa `.show()` atau memakai import `AlertDialog` yang salah	Panggil `.show()` dan impor `androidx.appcompat.app.AlertDialog`
Setelah Logout tetap langsung masuk Main	`apply()` belum dipanggil atau nama file berbeda	Pastikan `editor.apply()` dan nama `user_pref` sama di semua Activity
Fragment tidak tampil	Container belum di-constraint atau id tidak sama	Cek constraint `fragment_container` dan `binding.fragmentContainer.id`
Tombol back menampilkan layar kosong	Fragment awal ikut masuk back stack	Panggil `replaceFragment(SatuFragment(), addToBackStack = false)`

8 Ringkasan

Topik	Kata kunci	Fungsi
AlertDialog	`AlertDialog.Builder`, `setPositiveButton`, `show()`	Pesan dan konfirmasi ke pengguna
SharedPreferences	`getSharedPreferences`, `edit()`, `apply()`	Menyimpan data sederhana (key-value) secara permanen
Status login	`isLogin`, `clear()`, `remove()`	Menentukan halaman awal dan menghapus sesi saat Logout
Splash Screen	`lifecycleScope`, `delay()`	Layar pembuka sekaligus pengecekan sesi
Fragment	`Fragment()`, `onCreateView()`	Bagian UI yang bisa dipakai ulang di dalam satu Activity
Mengganti Fragment	`replace()`, `addToBackStack()`	Menampilkan fragment ke container dan kembali dengan tombol back